

Business Memo: Deployment Recommendation for Inventory Replenishment RL Agent

A tabular Q-learning agent was developed and evaluated for a 30-day inventory replenishment problem. Inventory managers are facing the challenges of determining the amount of inventories to be ordered each day in conditions of uncertain demand and delivery lead time of suppliers. Ordering excess inventories will lead to increased storage costs, whereas ordering too little causes stockouts and lost sales. Therefore, the goal of the Q-agent is to optimize inventory management by maximizing profit through balanced sales revenue, holding cost, stockout penalty, and ordering cost.

The environment represents a business scenario which operates in accordance with seven state variables (inventory level, day of week, 7-day rolling average demand, pending order quantity, lead time remaining, inventory position, and days remaining). Each day the agent chooses one out of seven discrete order quantities {0, 20, 40, 50, 60, 70, 80 units} to maximize daily profit: **Profit = Sales Revenue – Holding Cost – Stockout Penalty – Ordering Cost**

The demand follows a Poisson distribution with $\lambda=20$ on weekdays and $\lambda=25$ on weekends, with a 2-day replenishment lead time. The agent was trained over 3,000 episodes with $\alpha=0.1$, $\gamma=0.95$ and ϵ initially set at 1.0 decaying linearly to 0.05.

Recommendation

The results show that the Q-learning agent achieved a mean reward of \$5,521 per episode, while the reorder point policy ($s=60$, $S=120$) yielded an average reward of \$5,512 and random policy of \$4,675 per episode across 200 evaluation episodes. The grid-searched reorder-point policy ($s=70$, $S=130$) achieved \$5,573 per episode, slightly better than the Q-learning agent. However, the agent performed significantly better than all rule-based policies on stockout frequency, with 9.4 lost-sale events per episode compared to 15.5 for the standard reorder-point and 3.1 for the best-tuned variant, which clearly demonstrates a trade-off between average reward and service level stability.

Based on the test results, shadow deployment of the agent is recommended over immediate production deployment. According to the results, shadow deployment is suggested over production deployment. Although the agent performs well in the simulated environment, it has been found that the agent was vulnerable to reward hacking, demand surges, and distributional shift. These risks need to be monitored in production-like setting before taking any further steps to deploy the agent.

Performance Summary

Metric	Random (lower bound)	Reorder-Point ($s=60$ $S=120$)	Reorder-Point ($s=70$ $S=130$)	Q-Learning Agent
Mean episode reward	\$4,675	\$5,512	\$5,573	\$5,521
Std episode reward	\$499	\$222	\$262	\$257
Mean units sold / ep	622	625	638	632
Mean stockouts / ep	18.5	15.5	3.1	9.4
Mean ordering cost / ep	\$281	\$194	\$194	\$200
Mean holding cost / ep	\$1,118	\$456	\$574	\$521

The Q-learning agent achieved a mean episode reward of \$5,521, slightly exceeding the standard reorder point policy ($s = 60$, $S = 120$) that yielded \$5,512 and outperforming the random policy by \$846. Its most notable advantage compared to the standard operational baseline lies in service level, which was reduced to 9.4 stockouts per episode from an average of 15.5 per episode (39% improvement). As can be seen in Figure 1 (policy comparison), the agent also shows a reliable reward distribution, with its standard deviation (\$257) being almost equal to both reorder-point variants (\$222 and \$262), confirming the learned policy is stable rather than occasionally lucky.

However, the grid-searched reorder point policy ($s = 70$, $S = 130$) achieved the highest reward mean of \$5,573, surpassing the Q-learning agent by \$52 per episode with even fewer average stockouts, only 3.1 per episode. As depicted in Figure 2 (cost breakdown), this strategy attained such a significant improvement by maintaining a larger inventory buffer and therefore higher carrying cost (\$574 vs \$521). This policy aims to minimize risks, whereas the Q-learning agent targets leaner inventory profile that accepts more stockout risk in exchange for lower carrying costs. Whether this trade-off is better off will depend on how much weight the business places on storage costs versus service level consistency.

Operational Behavior

Figure 3 (agent behavior episode) shows that the agent places an order every 2–3 days at the quantity of 60–80 units throughout the 30-day horizon. Instead of waiting until stocks have been depleted, it maintains a proactively replenishing pace of making

large orders ahead of the weekend demand surge ($\lambda=25$) and smaller orders mid-week. The learning curve (Figure 4) demonstrates that this strategy has been learned: the 100-episode moving average increases from \$4,200 at episode 1 to a plateau of \$5,000 at episode 500, remaining consistent through episode 3,000.

Failure and Mitigation Analysis

1. Reward Hacking

By decreasing the stockout penalty from \$5 to \$1 per unit, a new trained agent avoided making any orders and accepted stockouts in 66.7% of the greedy evaluations since they became relatively cheaper than carrying inventory. This demonstrates that the agent is highly sensitive to reward configuration. If penalties are misaligned with real business objectives, the agent may learn behaviors that appear optimal from its perspective but is detrimental to business performance, leading to higher stockout rates, lost revenue, and reduced customer satisfaction. To address this, the stockout penalty should be reviewed and updated periodically based on the real-life lost sale cost estimates. An production alert should be triggered if inventory is completely depleted for more than three consecutive days.

2. Unsafe Behaviour Under Demand Surge (during promotional event or seasonal peak)

When tested on a demand function that was 50% above the training level, average stockouts rised from 10.7 to 287.7 units per episode, which is 27 times higher. The agent has no mechanism to detect out-of-distribution demand because the rolling average approach always lags behind sudden demand increases. To solve this, a demand-surge safety override could be considered. If the rolling average demand exceeds $1.3\times$ the training mean, the ordering strategy switches to the reorder-point policy until demand normalises. Quarterly retraining on recent demand data would also reduce this vulnerability over time.

3. Training Instability (Figure 5)

Using a learning rate of $\alpha=0.9$ produced a reward standard deviation of 1,002 across the final 200 training episodes, compared to 283 for the well-tuned $\alpha=0.1$ agent. The high learning rate prevents convergence because the Q-values kept on overshooting and oscillating. This poses a training-time risk that generate unreliable results upon retraining. Therefore, the hyperparameters $\alpha=0.1$, $\gamma=0.95$, and ϵ -decay=0.995 need to be versioned and gate-hold out for retraining before replacing the current ones.

4. Distributional Robustness

For an unseen pattern in which demand peaks on Mondays and Sundays with low demand mid-week, the agent achieved a reward of \$4,979 compared to \$4,886 for the reorder point policy, giving it an edge of \$93. This is a good outcome considering that the agent generalises beyond its training distribution and outperforms the rule-based baseline even on unseen demand patterns. Yet, the performance improvement is not substantial enough to eliminate the need for the demand-surge override and the proposed demand-surge override should remain in place as a safeguard. Future robustness could be improved through domain randomization (train on a mixture of demand distributions) or adding a safety override which activates the reorder-point policy when the average rolling demand deviates considerably from the training data.

Conclusion

The Q-learning agent achieved a mean episode reward of \$5,521, matching the standard reorder-point policy and reducing stockouts by 39%. Although the grid-searched reorder-point policy ($s=70$, $S=130$) is slightly better in raw profit (\$5,573), the service level stability that the agent maintains will bring more value through customer retention in the long run.

Failure analysis identified vulnerabilities to demand surges, reward misspecification, and distributional shift. These are manageable with the following measures: maintaining the reorder-point policy as an active fallback, logging all agent recommendations for auditability, tracking inventory service levels in production, retraining only after offline validation, and requiring human review for unusually large orders. These controls reduce the risk of unexpected behaviour while preserving the benefits of adaptive decision-making.

Final recommendation: shadow deploy

The Q-learning agent exhibited measurable improvement over both baseline policies and learned economically reasonable inventory behavior. Nevertheless, the performance advantage is rather modest, and several failure modes remain possible due to shifting demands. A shadow deployment allows the organization to compare agent recommendations against existing operations, test and monitor system behavior under realistic demands, and gather additional data for future model improvements. If the agent continues to outperform the reorder-point policy over a longer period of observations without causing service-level issues, it could be rolled out into production.

Figure 1: Policy Comparison

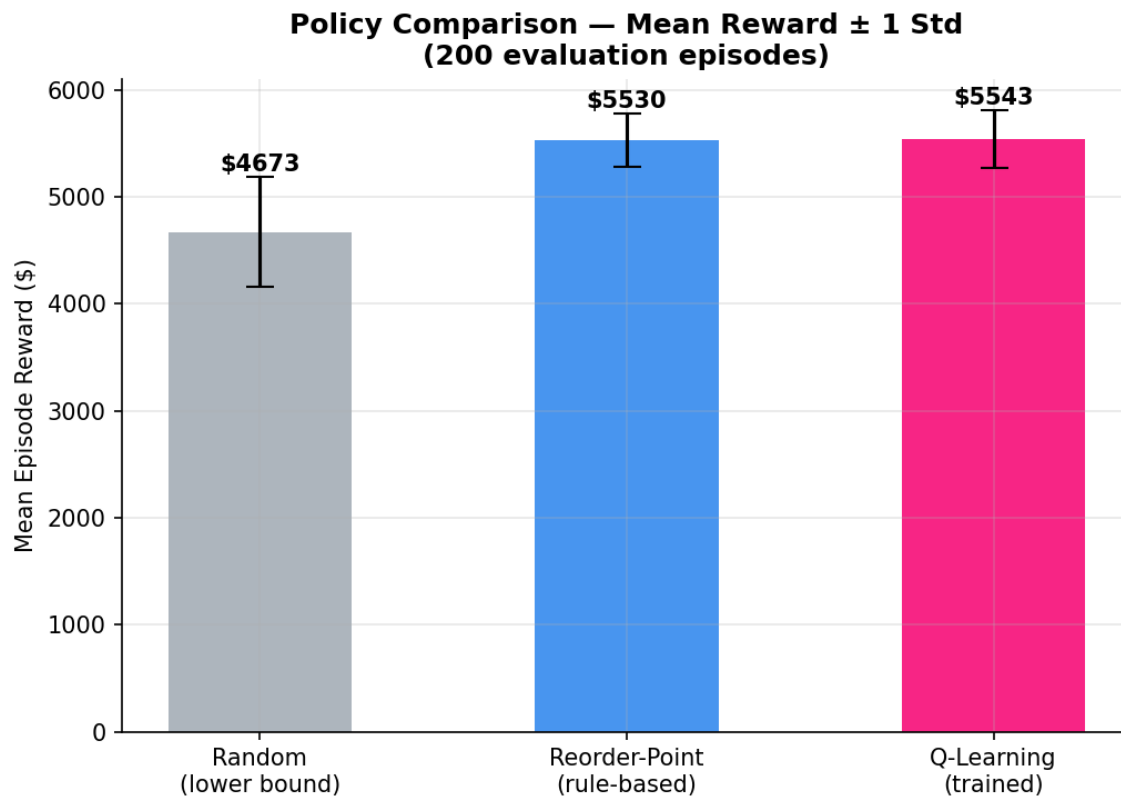


Figure 2: Cost Breakdown

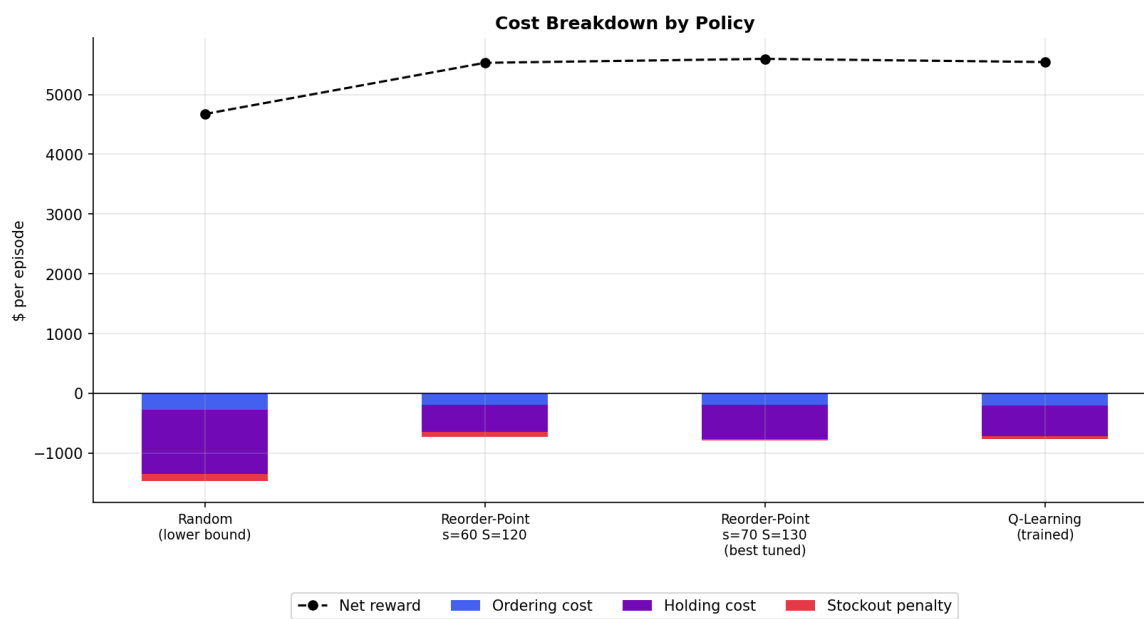


Figure 3: Agent Behavior Episode

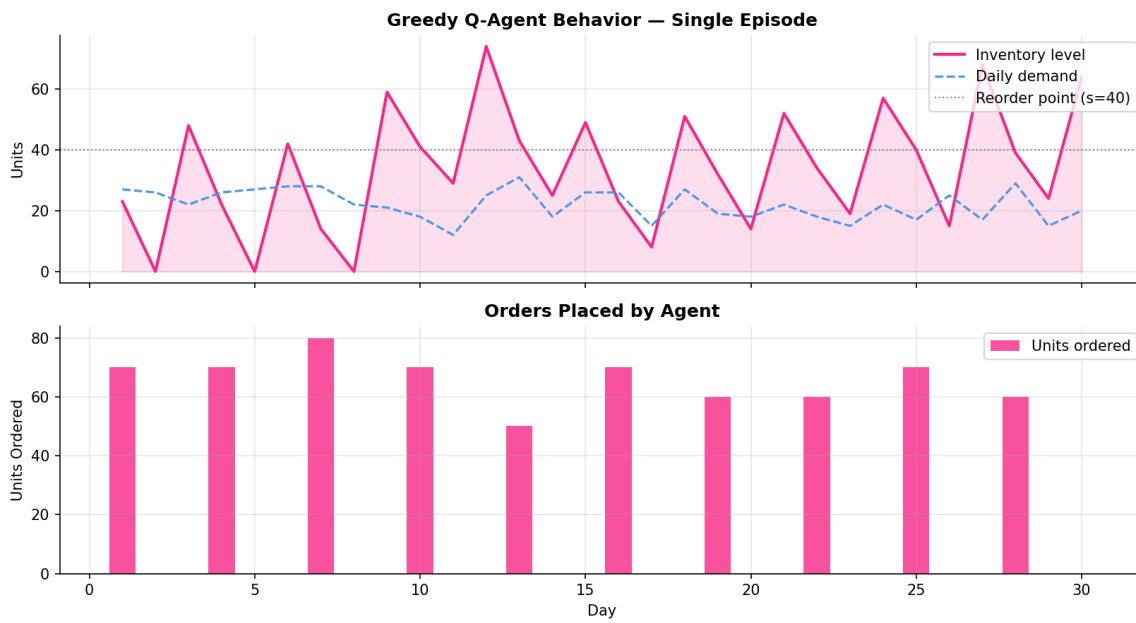


Figure 4: Training Curve

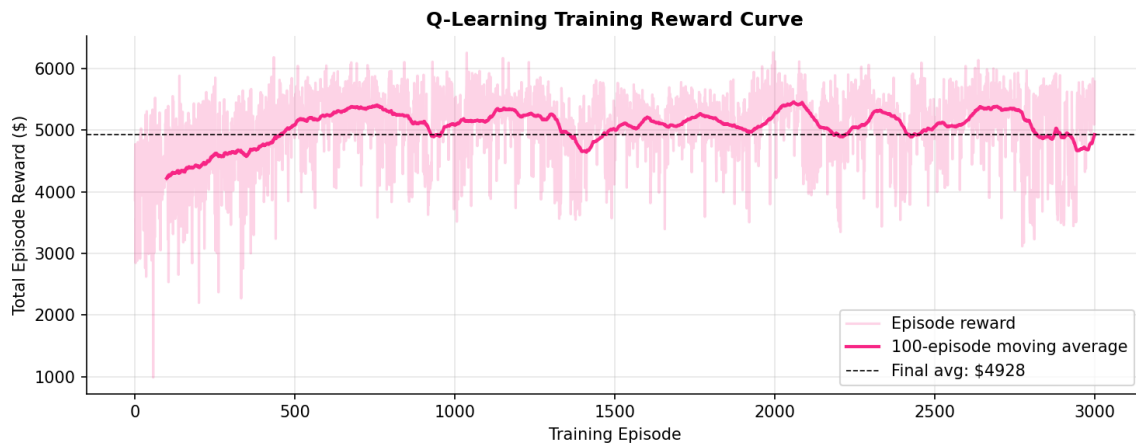


Figure 5: Training Instability

